

Object-Oriented Programming (OOP) Interview Q&A;

Comprehensive Interview Questions with Quick Points

1. What is a class?

- A blueprint or template for creating objects.
- Defines properties (attributes) and behaviors (methods).

2. What is an object?

- An instance of a class.
- Has state (attributes) and behavior (methods) defined by its class.
- Allocates memory when created.

3. What is the difference between a class and an object?

- Class is a logical entity; Object is a physical entity.
- Class doesn't consume memory; Object does.
- Class is declared once; Objects can be created multiple times.

4. Why do we create objects?

- To utilize the variables and methods defined inside a class.
- To represent real-world entities in our program.

5. Can we create multiple objects from a single class?

- Yes, you can create any number of objects from a single class.
- Each object has its own independent memory space for attributes.

6. Where are objects stored?

- In Python, objects are stored in heap memory.
- Variables that refer to them are stored in stack memory.

7. What is the purpose of the self keyword?

- Represents the current instance of the class.
- Used to access variables and methods associated with the current object.

8. Is self a keyword in Python?

- No, 'self' is just a widely followed convention, not a strict keyword.
- You could theoretically use any other name (like 'this' or 'me').

9. Why is self passed as the first parameter to instance methods?

- When calling obj.method(), Python internally translates it to Class.method(obj).
- It ensures the method knows which object's data it is operating on.

10. Can we rename self? If yes, should we?

- Yes, it can be renamed to anything.
- No, it is highly discouraged as it violates PEP 8 conventions and harms code readability.

11. What are the four pillars of Object-Oriented Programming (OOP)?

- Encapsulation

- Abstraction
- Inheritance
- Polymorphism

12. What is polymorphism?

- The ability of different objects to respond in their own way to the same method call.
- Example: `len()` works on strings, lists, and tuples differently.

13. What is Duck Typing in Python?

- 'If it walks like a duck and quacks like a duck, it must be a duck.'
- Python checks for the presence of methods/attributes at runtime rather than strict class types.

14. What is Dynamic Binding?

- Method calls are resolved at runtime rather than compile time.
- Python natively supports this since types are determined at runtime.

15. What is Operator Overloading?

- Giving extended meaning to standard operators like `+`, `-`, `*`, etc.
- Example: `+` adds numbers, but concatenates strings and lists. Implemented via dunder methods like `__add__`.

16. What is a constructor?

- A special method automatically called when an object is created.
- Used to initialize the state (instance variables) of the object.

17. What are the types of constructors in Python?

- Default Constructor (no arguments except `self`).
- Parameterized Constructor (takes additional arguments along with `self`).

18. Can a class have multiple constructors?

- No, Python does not support multiple constructors (method overloading).
- Defining multiple `__init__` methods will overwrite the previous ones. You can use default arguments instead.

19. What happens if a constructor is not defined?

- Python provides a default empty constructor internally.
- The object is still created, but no specific initialization occurs.

20. What is the difference between a constructor and a normal method?

- Constructor (`__init__`) is called automatically upon object creation.
- Normal methods must be called explicitly using the object reference.

21. Can a constructor return a value?

- No, `__init__` must only return `None`.
- Returning any other value raises a `TypeError`.

22. Can we call a constructor explicitly?

- Yes, by calling `object.__init__()`, but it's rarely needed.
- Usually called implicitly when instantiating: `obj = Class()`.

23. Can a constructor call another method?

- Yes, you can call any instance method using `self.method_name()` inside `__init__`.

24. What is constructor chaining?

- Calling a parent class's constructor from a child class's constructor.
- Done using `super().__init__()`.

25. What happens if the parent class has a parameterized constructor?

- The child class must explicitly call it and pass the required parameters using `super()`.

26. What is encapsulation?

- Wrapping data (variables) and methods into a single unit (class).
- Restricting direct access to some of the object's components.

27. Why do we need encapsulation?

- To protect an object's internal state from unintended modification.
- To improve maintainability and hide complexity.

28. Explain public, protected, and private members.

- Public: Accessible from anywhere.
- Protected (`_var`): Indicates internal use, accessible from subclasses (convention).
- Private (`__var`): Name mangled, inaccessible directly from outside (enforced).

29. Does Python truly support private variables?

- No, Python uses 'name mangling' for double underscore `__` variables.
- They can still be accessed via `_ClassName__variable`.

30. What is Name Mangling?

- Python automatically changes `__var` to `_ClassName__var` to prevent accidental access.
- Used to protect instance variables from being overridden in subclasses.

31. How can we access private variables?

- Using getter and setter methods.
- Or via name mangling: `obj._ClassName__private_var`.

32. What is the difference between encapsulation and abstraction?

- Encapsulation hides the internal data (implementation detail) using access modifiers.
- Abstraction hides the internal complexity and only shows essential features.

33. What are the advantages of encapsulation?

- Data Security / Hiding.
- Better control over data via getters/setters.
- Increased flexibility and maintainability.

34. What is inheritance?

- A mechanism where one class (child) acquires the properties and methods of another class (parent).

35. Why do we use inheritance?

- For Code Reusability.
- To establish a logical 'is-a' relationship (e.g., Dog is-a Animal).

36. What are the different types of inheritance?

- Single
- Multiple
- Multilevel
- Hierarchical
- Hybrid

37. Does Python support multiple inheritance?

- Yes, a class can inherit from more than one base class.
- Example: `class Child(Parent1, Parent2):`

38. What is multilevel inheritance?

- A class inherits from a child class, forming a chain (Grandparent -> Parent -> Child).

39. What is hierarchical inheritance?

- Multiple child classes inherit from a single parent class.

40. What is hybrid inheritance?

- A combination of two or more types of inheritance (e.g., Multiple + Multilevel).

41. What is the difference between inheritance and composition?

- Inheritance is an 'is-a' relationship.
- Composition is a 'has-a' relationship (class contains an object of another class).
- Composition is often preferred for loose coupling.

42. What is a real-life example of inheritance?

- Vehicle (Parent) -> Car (Child).
- Employee (Parent) -> Manager (Child).

43. When should inheritance NOT be used?

- When there is no clear 'is-a' relationship.
- If you only need a couple of methods from a class, use composition instead.

44. Why can multiple inheritance become complicated?

- It can lead to ambiguity if multiple parent classes have methods with the same name.
- This is known as the Diamond Problem.

45. What is Method Resolution Order (MRO)?

- The order in which Python searches for base classes when executing a method.
- Can be viewed using `ClassName.__mro__` or `ClassName.mro()`.

46. What is the Diamond Problem?

- An ambiguity that arises when two classes B and C inherit from A, and class D inherits from both B and C.
- If D calls a method defined in A (and overridden in B and C), which one should it use?

47. How does Python solve the Diamond Problem?

- Python solves it using MRO (Method Resolution Order).
- It follows the C3 Linearization algorithm to determine the exact order.

48. Explain C3 Linearization.

- An algorithm used to calculate MRO in Python.

- Ensures a child is checked before its parents, and multiple parents are checked left-to-right.

49. What is method overriding?

- When a child class provides a specific implementation of a method already defined in its parent class.
- Must have the same name and parameters.

50. Why is method overriding used?

- To provide a specialized behavior for a method inherited from a parent class.
- Essential for runtime polymorphism.

51. What is the difference between method overriding and method overloading?

- Overriding: Same method name and parameters in parent and child classes.
- Overloading: Same method name but different parameters within the same class (not natively supported in Python).

52. Does Python support method overloading?

- Not natively. Defining a method twice will just overwrite the first one.
- Can be simulated using default arguments or `*args/**kwargs`.

53. What happens if a child class doesn't override a method?

- It simply uses the implementation provided by the parent class.

54. Can constructors be overridden?

- Yes, if a child class defines `__init__`, it overrides the parent's `__init__`.
- The parent's constructor can still be called using `super().__init__()`.

55. Can private methods be overridden?

- No, private methods (`__method`) are name-mangled and specific to the class.
- They are not inherited, so they cannot be overridden.

56. What is runtime polymorphism?

- Achieved via Method Overriding.
- The exact method to execute is determined at runtime based on the object's actual type, not the reference type.

57. What is `super()`?

- A built-in function that returns a proxy object allowing you to call methods of the parent class.
- Commonly used to invoke parent constructors.

58. Why do we use `super()`?

- To access inherited methods that have been overridden in a class.
- To ensure cooperative multiple inheritance.

59. Can `super()` be used to access parent variables?

- Yes, it can access class variables of the parent.
- However, it is generally used for method calls.

60. Can we use the parent class name instead of `super()`?

- Yes, like `ParentClass.method(self)`.
- However, `super()` is preferred as it handles multiple inheritance and MRO automatically.

61. How does `super()` work in multiple inheritance?

- It strictly follows the MRO (Method Resolution Order).
- It calls the next class in the MRO chain, which might not be the direct parent.

62. What are the advantages of using `super()`?

- Avoids hardcoding parent class names.
- Works seamlessly with multiple inheritance.

63. What is abstraction?

- Hiding implementation details and exposing only the essential features to the user.
- Focuses on 'what' an object does rather than 'how' it does it.

64. What is the difference between abstraction and encapsulation?

- Abstraction hides complexity (design level).
- Encapsulation hides data (implementation level).

65. What is an abstract class?

- A class that contains one or more abstract methods.
- Cannot be instantiated directly.

66. Can we create an object of an abstract class?

- No, attempting to instantiate it will raise a `TypeError`.

67. What is an abstract method?

- A method that is declared but contains no implementation.
- Must be overridden by all concrete child classes.

68. Why do we use abstract classes?

- To define a strict blueprint for other classes to follow.
- Ensures that certain methods are implemented by subclasses.

69. Which module provides abstract class support?

- The ``abc`` module (Abstract Base Classes).
- You inherit from `ABC` and use the `@abstractmethod` decorator.

70. What is the difference between an interface and an abstract class?

- Python doesn't have strict interfaces.
- An interface only contains abstract methods. An abstract class can have both abstract and concrete methods.

71. Can an abstract class have constructors, variables, and concrete methods?

- Yes, it can have full implementations alongside abstract methods.

72. When should we choose an abstract class?

- When you want to share code among closely related classes but still enforce certain methods to be implemented.

73. What is an exception?

- An event that disrupts the normal flow of the program during runtime.
- Example: `ZeroDivisionError`, `FileNotFoundError`.

74. What is the difference between a syntax error and an exception?

- Syntax Error: Invalid code structure, fails before execution (e.g., missing colon).
- Exception: Valid code but fails during execution (e.g., dividing by zero).

75. What is the purpose of the try block?

- To enclose code that might potentially raise an exception.

76. What is the purpose of the except block?

- To catch and handle the exception raised in the try block, preventing program crash.

77. What is the purpose of the else block?

- Executes only if no exception was raised in the try block.
- Useful for code that must run only on success.

78. What is the purpose of the finally block?

- Executes unconditionally, regardless of whether an exception occurred or not.
- Used for clean-up actions like closing files or network connections.

79. When does the finally block execute?

- Always. Even if the try block has a return statement or raises an unhandled exception.

80. What is the difference between raise and assert?

- `raise` is used to manually trigger an exception.`
- `assert` is used for debugging to check if a condition is True; raises AssertionError if False.`

81. Can we write multiple except blocks?

- Yes, a single try block can have multiple except blocks to handle different exceptions differently.

82. What is the difference between Exception and BaseException?

- BaseException is the root of all exceptions (including SystemExit, KeyboardInterrupt).
- Exception is a subclass of BaseException and the base for all regular, non-system-exiting errors.

83. What is the difference between an Error and an Exception?

- In Python, they are technically the same (both inherit from Exception).
- Conceptually, errors might be fatal, while exceptions are recoverable.

84. What is a custom exception?

- A user-defined exception class created by inheriting from Python's Exception class.

85. Why do we create custom exceptions?

- To provide more meaningful and domain-specific error messages.
- E.g., InsufficientFundsError for a banking app.

86. What is the difference between ValueError and TypeError?

- ValueError: Right type, but inappropriate value (e.g., `int('xyz')`).
- TypeError: Operation applied to inappropriate type (e.g., `'a' + 5`).

87. Why should we catch specific exceptions instead of using a generic except?

- Catching generic exceptions hides bugs and makes debugging extremely difficult.
- Always catch specific expected exceptions (e.g., `FileNotFoundError`).

88. Why do we use files?

- To store data permanently in non-volatile memory (like a hard drive).
- Variables are stored in RAM and lost when the program ends.

89. What are the different file opening modes?

- r: Read
- w: Write
- a: Append
- x: Exclusive create
- b: Binary
- t: Text
- +: Updating (read/write).

90. What is the difference between r and r+?

- r: Only allows reading.
- r+: Allows both reading and writing (pointer starts at the beginning).

91. What is the difference between w and a?

- w: Overwrites the file if it exists, creates it if it doesn't.
- a: Appends data to the end of the file, preserving existing content.

92. What is the difference between text mode and binary mode?

- Text mode ('t'): Handles strings, performs platform-specific newline translations.
- Binary mode ('b'): Handles bytes directly (used for images, PDFs, etc.).

93. What does the seek() method do?

- Moves the file pointer to a specific byte position within the file.

94. What does the tell() method do?

- Returns the current byte position of the file pointer.

95. What is the difference between read(), readline(), and readlines()?

- read(): Reads the entire file as a single string.
- readline(): Reads one line at a time.
- readlines(): Reads all lines and returns a list of strings.

96. Why should we use the with statement?

- It acts as a context manager.
- Automatically closes the file when the block is exited, even if an exception occurs.

97. What happens if a file is not closed?

- Memory leaks.
- Data might not be fully flushed to disk (lost data).
- The file might remain locked by the OS.

98. What is buffering in file handling?

- Storing data temporarily in RAM before flushing it to the disk to improve performance.

99. What is a text file?

- A file containing human-readable characters, organized into lines.

100. What is a binary file?

- A file containing raw bytes (e.g., compiled code, images, audio). Not human-readable.

101. What is the difference between a CSV file and a TXT file?

- CSV stores structured tabular data separated by commas.
- TXT is for unstructured plain text.

102. What are the advantages of file handling?

- Data persistence across multiple runs.
- Allows processing large datasets that don't fit in RAM.

103. What is serialization?

- Converting an object's state in memory into a byte stream (or JSON string) for storage or transmission.

104. What is deserialization?

- Reconstructing an object in memory from a byte stream or string.

105. Why is serialization required?

- To save the state of an object to a database/file.
- To send objects over a network (like in REST APIs).

106. What are the advantages of serialization?

- Data persistence.
- Platform-independent data transfer (especially with JSON).

107. What is the difference between Pickle and JSON?

- Pickle: Python-specific, binary format, can serialize complex Python objects, insecure against untrusted data.
- JSON: Language-independent, text format, secure, only supports basic data types (dicts, lists, strings, numbers).

108. Why is JSON preferred over Pickle in many applications?

- JSON is human-readable, secure, and easily consumed by other languages (like JavaScript/Java).

109. What are the limitations of Pickle?

- It is a security risk if unpickling untrusted data (can execute arbitrary code).
- Pickled files from Python 3 might not open in Python 2 (version specific).
- Not understood by other programming languages.

110. Can Pickle serialize every Python object?

- No. It cannot serialize things like open file handles, database connections, or network sockets.

111. Where is serialization commonly used?

- REST APIs (JSON), caching (Redis/Pickle), saving game states, Machine Learning model deployment (Pickle/Joblib).

112. Explain a real-world use case of serialization.

- Sending data from a Python backend server to a React frontend web application using JSON.

113. What is the difference between `isinstance()` and `type()`?

- `isinstance()` considers inheritance (checks if object is instance of class or subclass).
- `type()` only checks the exact class, ignoring inheritance.

114. When should you use `isinstance()` instead of `type()`?

- Usually always. `isinstance()` is more robust because it respects polymorphism and inheritance.

115. What is the purpose of `issubclass()`?

- Checks if a given class is a direct or indirect subclass of another class.

116. What is the difference between `dir()` and `help()`?

- `dir()`: Returns a list of valid attributes and methods of an object.
- `help()`: Prints the detailed documentation (docstrings) for an object or module.

117. How do you inspect an object's attributes and methods?

- Using built-in functions like `dir(obj)`, `vars(obj)`, `getattr()`, or the `inspect` module.